

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int pid;

    pid = fork();

    if(pid < 0) {
        printf("CHILD PROCESS NOT CREATED\n");
        exit(1);
    }

    if(pid == 0) {
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp("ls", "ls", NULL);

        printf("Execution failed\n");
    }
    else {
        printf("\n--- PARENT PROCESS ---\n");
        printf("Parent PID: %d\n", getpid());
        printf("Parent's Parent PID: %d\n",
            getppid());
    }

    printf("\nIT CAN BE EXECUTED child and
        parent\n");

    return 0;
}

```

```

--- CHILD PROCESS ---
--- PARENT PROCESS ---
Child PID: 16586
Parent PID: 16585
Parent PID: 16585
Parent's Parent PID: 16578

```

IT CAN BE EXECUTED child and parent

=== Code Execution Successful ===

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main() {
    int shmid;
    char *str;

    // 1. Set size
    int size = 1024;

    // 2. Access shared memory
    shmid = shmget((key_t)1234, size, 0666);

    if (shmid < 0) {
        printf("Shared memory access failed\n");
        return 1;
    }

    // 3. Attach
    str = (char*) shmat(shmid, NULL, 0);

    // 4. Read data
    printf("Data received from shared memory: %s\n", str);

    // 5. Detach
    shmdt(str);

    return 0;
}
```

Sender Output:

Data written to shared memory: Hello from Sender Process!

Receiver Output:

Data received from shared memory: Hello from Sender Process!

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
#include <unistd.h>

int main() {
    int shmid;
    char *str;

    // 1. Set size
    int size = 1024;

    // 2. Allocate shared memory
    shmid = shmget((key_t)1234, size, 0666 | IPC_CREAT);

    if (shmid < 0) {
        printf("Shared memory creation failed\n");
        return 1;
    }

    // 3. Attach
    str = (char*) shmat(shmid, NULL, 0);

    // 4. Write data
    sprintf(str, "Hello from Sender Process!");

    printf("Data written to shared memory: %s\n", str);

    // 5. Delay
    sleep(10);

    // 6. Detach
    shmdt(str);

    return 0;
}
```

Produced: 1

Consumed: 1

Produced: 2

Consumed: 2

Produced: 3

Consumed: 3

Produced: 4

Consumed: 4

Produced: 5

Consumed: 5

Produced: 6

Consumed: 6

Produced: 7

Consumed: 7

Produced: 8

Consumed: 8

Produced: 9

Consumed: 9

Produced: 10

Consumed: 10


```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

// Semaphores
sem_t empty, full, mutex;

// Producer Thread
void* producer(void* arg) {
    for(int i = 1; i <= 10; i++) {

        // Wait for empty slot
        sem_wait(&empty);

        // Enter critical section
        sem_wait(&mutex);

        // Produce item
        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;

        // Exit critical section
        sem_post(&mutex);

        // Increase full count
        sem_post(&full);

        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

    // 1. Initialize semaphores
    sem_init(&empty, 0, BUFFER_SIZE); // initially all slots empty
    sem_init(&full, 0, 0);             // initially no items
    sem_init(&mutex, 0, 1);            // binary semaphore

    // 2. Create threads
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    // 3. Wait for threads to finish
    pthread_join(p, NULL);
    pthread_join(c, NULL);

    // Destroy semaphores
    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}

```